

Multi-Tenancy System Design

Complete guide to designing multi-tenant systems with microservices, Gen AI/LLM, RAG, and monitoring.

What is Multi-Tenancy?

Multi-tenancy = Single application instance serves multiple customers (tenants)

Key Concept: Each tenant's data and configuration are isolated, but they share the same infrastructure.

Examples:

- Salesforce: Each company is a tenant
- Slack: Each workspace is a tenant
- Gmail: Each organization's Google Workspace is a tenant
- Shopify: Each store is a tenant

Benefits:

- Lower infrastructure costs (shared resources)
- Easier maintenance (single codebase)
- Faster deployment (update once, affects all)
- Better resource utilization

Challenges:

- Data isolation (security)
- Noisy neighbor problem (performance)
- Customization per tenant
- Tenant-specific monitoring

Multi-Tenancy Patterns

Pattern 1: Database Per Tenant (Highest Isolation)

What it is:

- Each tenant has their own database
- Complete data isolation
- Can be on different database servers

Architecture:

```
Tenant A → App → Database A
Tenant B → App → Database B
Tenant C → App → Database C
```

Implementation:

```
# Tenant resolver
def get_database_connection(tenant_id):
    database_map = {
        'tenant_a': 'postgresql://db1.example.com/tenant_a',
        'tenant_b': 'postgresql://db2.example.com/tenant_b',
        'tenant_c': 'postgresql://db3.example.com/tenant_c'
    }
    return database_map[tenant_id]

# Middleware
@app.middleware("http")
async def tenant_middleware(request: Request, call_next):
    tenant_id = extract_tenant_id(request) # From subdomain, header, JWT
    request.state.db = get_database_connection(tenant_id)
    response = await call_next(request)
    return response

# Usage
@app.get("/users")
def get_users(request: Request):
    db = request.state.db
    users = db.query("SELECT * FROM users")
    return users
```

Pros: ☒ Complete data isolation ☒ Easy to backup/restore per tenant ☒ Can customize schema per tenant
☒ Easy to migrate tenant to different infrastructure ☒ Compliance friendly (data residency)

Cons: ☒ Higher infrastructure cost ☒ More databases to manage ☒ Harder to aggregate cross-tenant analytics ☒ Schema migrations need to run on all DBs

When to use:

- High security/compliance requirements
- Tenants need custom schemas
- Large enterprise customers
- Different data residency requirements

Pattern 2: Shared Database with Row-Level Isolation

What it is:

- Single database for all tenants
- Each row has tenant_id column
- Filter by tenant_id in all queries

Architecture:

Tenant A ↘
 Tenant B → App → Single Database (with tenant_id column)
 Tenant C ↗

Implementation:

```
# Database model
class User(Base):
    __tablename__ = 'users'

    id = Column(Integer, primary_key=True)
    tenant_id = Column(String, nullable=False, index=True)
    name = Column(String)
    email = Column(String)

# Automatic tenant filtering
@app.middleware("http")
async def tenant_middleware(request: Request, call_next):
    tenant_id = extract_tenant_id(request)
    request.state.tenant_id = tenant_id
    response = await call_next(request)
    return response

# Query with tenant filter
@app.get("/users")
def get_users(request: Request, db: Session):
    tenant_id = request.state.tenant_id
    users = db.query(User).filter(User.tenant_id == tenant_id).all()
    return users

# Create with tenant
@app.post("/users")
def create_user(user: UserCreate, request: Request, db: Session):
    tenant_id = request.state.tenant_id
    new_user = User(
        tenant_id=tenant_id,
        name=user.name,
        email=user.email
    )
    db.add(new_user)
    db.commit()
    return new_user
```

Row-Level Security (PostgreSQL):

```
-- Enable RLS
ALTER TABLE users ENABLE ROW LEVEL SECURITY;

-- Policy: Users can only see their tenant's data
```

```
CREATE POLICY tenant_isolation ON users
  USING (tenant_id = current_setting('app.current_tenant')::text);

-- Set tenant in session
SET app.current_tenant = 'tenant_a';

-- All queries automatically filtered
SELECT * FROM users; -- Only returns tenant_a's users
```

Pros: ☒ Low infrastructure cost ☒ Easy to manage (single database) ☒ Easy cross-tenant analytics ☒ Single schema migration

Cons: ☒ Risk of data leakage (if tenant_id forgotten in query) ☒ Noisy neighbor (one tenant can impact others) ☒ Limited customization per tenant ☒ Harder to scale individual tenants

When to use:

- Many small tenants
- Cost-sensitive
- Standard features for all tenants
- Lower compliance requirements

Pattern 3: Hybrid (Schema Per Tenant)

What it is:

- Single database
- Each tenant has their own schema/namespace
- Provides middle-ground isolation

Architecture:

```
Tenant A → App → Database
                  ├── Schema A (tables for tenant A)
                  ├── Schema B (tables for tenant B)
                  └── Schema C (tables for tenant C)
```

Implementation (PostgreSQL):

```
# Create schema per tenant
def create_tenant_schema(tenant_id):
    with engine.connect() as conn:
        conn.execute(f"CREATE SCHEMA IF NOT EXISTS {tenant_id}")
        conn.execute(f"""
            CREATE TABLE {tenant_id}.users (
                id SERIAL PRIMARY KEY,
                name VARCHAR(100),
                email VARCHAR(100)
```

```

    """
)

# Switch schema per request
@app.middleware("http")
async def tenant_middleware(request: Request, call_next):
    tenant_id = extract_tenant_id(request)
    request.state.tenant_id = tenant_id

    # Set search path to tenant schema
    with engine.connect() as conn:
        conn.execute(f"SET search_path TO {tenant_id}")

    response = await call_next(request)
    return response

# Queries use active schema
@app.get("/users")
def get_users(db: Session):
    # Automatically queries from current schema (tenant)
    users = db.query(User).all()
    return users

```

Pros: ☒ Good isolation (schema-level) ☒ Lower cost than database-per-tenant ☒ Can customize schema per tenant ☒ Easier backup per tenant than row-level

Cons: ☒ Limited by database connection limits ☒ Schema migrations on all schemas ☒ Still single database (scaling limits)

When to use:

- Medium-sized tenants (10-1000)
- Need some customization
- Balance between cost and isolation

Multi-Tenancy in Microservices

Architecture Patterns

Pattern A: Tenant ID Propagation

```

API Gateway
├─ Extract tenant_id from request
├─ Add to header: X-Tenant-ID
└─ Forward to services

Service A
├─ Read X-Tenant-ID from header
├─ Filter data by tenant_id
└─ Forward to Service B (with header)

```

Service B

- └ Read X-Tenant-ID from header
- └ Filter data by tenant_id

Implementation:

```
# API Gateway
@app.middleware("http")
async def tenant_gateway(request: Request, call_next):
    # Extract tenant from subdomain, JWT, or header
    tenant_id = extract_tenant_from_subdomain(request.url.hostname)
    # Or from JWT
    # tenant_id = decode_jwt(request.headers.get("Authorization"))["tenant_id"]

    # Add to headers
    request.headers.__dict__["_list"].append(
        (b"x-tenant-id", tenant_id.encode()))

    response = await call_next(request)
    return response

# Microservice
@app.get("/orders")
def get_orders(request: Request):
    tenant_id = request.headers.get("X-Tenant-ID")

    # Forward to another service
    response = httpx.get(
        "http://inventory-service/stock",
        headers={"X-Tenant-ID": tenant_id}
    )

    return orders
```

Pattern B: Service Per Tenant (Dedicated Infrastructure)

Tenant A → Service Instance A → Database A
Tenant B → Service Instance B → Database B
Tenant C → Service Instance C → Database C

Kubernetes Implementation:

```
# Namespace per tenant
apiVersion: v1
kind: Namespace
```

```
metadata:
  name: tenant-a

---

# Deployment per tenant
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api-service
  namespace: tenant-a
spec:
  replicas: 2
  selector:
    matchLabels:
      app: api-service
      tenant: tenant-a
  template:
    metadata:
      labels:
        app: api-service
        tenant: tenant-a
    spec:
      containers:
        - name: api
          image: myapp:latest
          env:
            - name: TENANT_ID
              value: "tenant-a"
            - name: DATABASE_URL
              value: "postgresql://db-tenant-a:5432/tenant_a"
```

Pros: ☒ Complete isolation ☒ Independent scaling per tenant ☒ Different versions possible per tenant ☒
No noisy neighbor

Cons: ☒ High infrastructure cost ☒ Complex management (many services) ☒ Harder to update all tenants

When to use:

- Enterprise customers paying premium
- Need guaranteed resources
- Custom versions per tenant
- Compliance requires isolation

Multi-Tenancy with Gen AI / LLM

Challenge: LLM APIs are expensive and tenant-specific

Tenant-Specific Requirements:

- Different LLM models per tenant (Groq, OpenAI, Anthropic)
- Different prompts/instructions per tenant

- Usage quotas per tenant
- Cost tracking per tenant

Pattern 1: Tenant-Aware LLM Gateway

```

from dataclasses import dataclass
from typing import Dict
import anthropic
import groq

@dataclass
class TenantLLMConfig:
    provider: str # "openai", "anthropic", "groq"
    model: str
    api_key: str
    max_tokens: int
    temperature: float
    system_prompt: str
    monthly_quota: int # tokens

# Tenant configurations
TENANT_CONFIGS: Dict[str, TenantLLMConfig] = {
    "tenant_a": TenantLLMConfig(
        provider="groq",
        model="llama-3.3-70b",
        api_key="gsk_...",
        max_tokens=1000,
        temperature=0.7,
        system_prompt="You are a helpful assistant for Acme Corp.",
        monthly_quota=1_000_000
    ),
    "tenant_b": TenantLLMConfig(
        provider="anthropic",
        model="claude-3-5-sonnet-20241022",
        api_key="sk-ant-...",
        max_tokens=2000,
        temperature=0.5,
        system_prompt="You are a technical support agent for TechCo.",
        monthly_quota=5_000_000
    )
}

# Usage tracking
tenant_usage: Dict[str, int] = {}

async def llm_generate(tenant_id: str, prompt: str):
    config = TENANT_CONFIGS[tenant_id]

    # Check quota
    used = tenant_usage.get(tenant_id, 0)
    if used >= config.monthly_quota:
        raise Exception(f"Tenant {tenant_id} exceeded monthly quota")

```

```

# Route to appropriate provider
if config.provider == "groq":
    client = groq.AsyncGroq(api_key=config.api_key)
    response = await client.chat.completions.create(
        model=config.model,
        messages=[
            {"role": "system", "content": config.system_prompt},
            {"role": "user", "content": prompt}
        ],
        max_tokens=config.max_tokens,
        temperature=config.temperature
    )
    result = response.choices[0].message.content
    tokens_used = response.usage.total_tokens

elif config.provider == "anthropic":
    client = anthropic.AsyncAnthropic(api_key=config.api_key)
    response = await client.messages.create(
        model=config.model,
        max_tokens=config.max_tokens,
        temperature=config.temperature,
        system=config.system_prompt,
        messages=[{"role": "user", "content": prompt}]
    )
    result = response.content[0].text
    tokens_used = response.usage.input_tokens + response.usage.output_tokens

# Track usage
tenant_usage[tenant_id] = tenant_usage.get(tenant_id, 0) + tokens_used

# Log for billing
await log_llm_usage(tenant_id, config.provider, tokens_used)

return result

# API endpoint
@app.post("/api/chat")
async def chat(request: Request, message: str):
    tenant_id = request.state.tenant_id
    response = await llm_generate(tenant_id, message)
    return {"response": response}

```

Pattern 2: Tenant-Specific RAG (Retrieval-Augmented Generation)

Challenge: Each tenant has their own knowledge base

Architecture:

Tenant A → Query
↓

```

RAG System
├─ Vector Store A (Tenant A docs)
├─ Vector Store B (Tenant B docs)
└─ Vector Store C (Tenant C docs)
↓
Retrieve relevant docs for Tenant A
↓
LLM with Tenant A context
↓
Response

```

Implementation:

```

from langchain.embeddings import OpenAIEmbeddings
from langchain.vectorstores import Pinecone
from langchain.chat_models import ChatOpenAI
from langchain.chains import RetrievalQA
import pinecone

# Initialize
pinecone.init(api_key="...", environment="us-west1-gcp")

class TenantRAG:
    def __init__(self, tenant_id: str):
        self.tenant_id = tenant_id
        self.embeddings = OpenAIEmbeddings()

        # Namespace per tenant in Pinecone
        self.vectorstore = Pinecone.from_existing_index(
            index_name="multi-tenant-kb",
            embedding=self.embeddings,
            namespace=tenant_id # Isolate by namespace
        )

        # Tenant-specific LLM config
        config = TENANT_CONFIGS[tenant_id]
        self.llm = ChatOpenAI(
            model=config.model,
            api_key=config.api_key,
            temperature=config.temperature
        )

        # Create RAG chain
        self.qa_chain = RetrievalQA.from_chain_type(
            llm=self.llm,
            chain_type="stuff",
            retriever=self.vectorstore.as_retriever(
                search_kwargs={"k": 5} # Top 5 relevant docs
            )
        )

    async def query(self, question: str):

```

```

        # Retrieves from tenant's namespace only
        response = await self.qa_chain.arun(question)
        return response

# Usage
@app.post("/api/rag/query")
async def rag_query(request: Request, question: str):
    tenant_id = request.state.tenant_id

    rag = TenantRAG(tenant_id)
    answer = await rag.query(question)

    return {"answer": answer}

# Ingest documents per tenant
@app.post("/api/rag/ingest")
async def ingest_documents(
    request: Request,
    documents: List[str]
):
    tenant_id = request.state.tenant_id

    # Embed and store in tenant's namespace
    embeddings = OpenAIEmbeddings()
    Pinecone.from_texts(
        texts=documents,
        embedding=embeddings,
        index_name="multi-tenant-kb",
        namespace=tenant_id # Isolate by tenant
    )

    return {"status": "success", "count": len(documents)}

```

Alternative: Separate Vector DB per Tenant

```

# Tenant-specific Chroma collections
def get_tenant_vectorstore(tenant_id: str):
    import chromadb

    client = chromadb.PersistentClient(path=f"./chroma_db/{tenant_id}")
    collection = client.get_or_create_collection(f"kb_{tenant_id}")

    return collection

# Complete isolation
tenant_a_db = get_tenant_vectorstore("tenant_a")
tenant_b_db = get_tenant_vectorstore("tenant_b")

```

Pattern 3: Multi-Tenant LLM Evaluation

Challenge: Track quality metrics per tenant

```

from ragas import evaluate
from ragas.metrics import faithfulness, answer_relevancy, context_precision
from datasets import Dataset

class TenantEvaluator:
    def __init__(self, tenant_id: str):
        self.tenant_id = tenant_id
        self.rag = TenantRAG(tenant_id)

    async def evaluate_qa_quality(self, test_cases: List[Dict]):
        """
        Evaluate RAG quality for specific tenant
        """
        results = []

        for test in test_cases:
            question = test["question"]
            ground_truth = test["ground_truth"]

            # Get RAG response
            answer = await self.rag.query(question)

            # Get contexts used
            docs = self.rag.vectorstore.similarity_search(question, k=5)
            contexts = [doc.page_content for doc in docs]

            results.append({
                "question": question,
                "answer": answer,
                "contexts": contexts,
                "ground_truth": ground_truth
            })

        # Evaluate with RAGAS
        dataset = Dataset.from_list(results)
        scores = evaluate(
            dataset,
            metrics=[
                faithfulness,
                answer_relevancy,
                context_precision
            ]
        )

        # Store per-tenant metrics
        await store_tenant_metrics(self.tenant_id, scores)

        return scores

# Store metrics per tenant
async def store_tenant_metrics(tenant_id: str, scores: dict):

```

```
import datetime

metrics = {
    "tenant_id": tenant_id,
    "timestamp": datetime.datetime.now(),
    "faithfulness": scores["faithfulness"],
    "answer_relevancy": scores["answer_relevancy"],
    "context_precision": scores["context_precision"]
}

# Store in database
db.collection("tenant_metrics").insert_one(metrics)

# API to get tenant's RAG quality
@app.get("/api/metrics/rag-quality")
async def get_rag_quality(request: Request):
    tenant_id = request.state.tenant_id

    # Get latest metrics
    metrics = db.collection("tenant_metrics")\
        .find({"tenant_id": tenant_id})\
        .sort("timestamp", -1)\
        .limit(1)\
        .next()

    return metrics
```

Dashboard per Tenant:

```
# Grafana dashboard per tenant
@app.get("/api/metrics/dashboard")
async def tenant_dashboard(request: Request):
    tenant_id = request.state.tenant_id

    # Aggregate metrics
    metrics = {
        "llm_requests": get_llm_request_count(tenant_id),
        "tokens_used": get_token_usage(tenant_id),
        "avg_latency": get_avg_latency(tenant_id),
        "rag_quality": get_rag_quality_score(tenant_id),
        "error_rate": get_error_rate(tenant_id),
        "cost": calculate_cost(tenant_id)
    }

    return metrics
```

Multi-Tenant Monitoring

Prometheus Metrics with Tenant Labels

```
from prometheus_client import Counter, Histogram, Gauge

# Metrics with tenant_id label
llm_requests = Counter(
    'llm_requests_total',
    'Total LLM requests',
    ['tenant_id', 'model', 'status']
)

llm_latency = Histogram(
    'llm_request_duration_seconds',
    'LLM request latency',
    ['tenant_id', 'model']
)

llm_tokens = Counter(
    'llm_tokens_total',
    'Total tokens used',
    ['tenant_id', 'model', 'type']
)

rag_quality = Gauge(
    'rag_quality_score',
    'RAG quality metrics',
    ['tenant_id', 'metric']
)

# Track metrics
async def llm_generate_with_metrics(tenant_id: str, prompt: str):
    config = TENANT_CONFIGS[tenant_id]

    with llm_latency.labels(tenant_id=tenant_id, model=config.model).time():
        try:
            response = await llm_generate(tenant_id, prompt)

            # Success
            llm_requests.labels(
                tenant_id=tenant_id,
                model=config.model,
                status='success'
            ).inc()

            return response

        except Exception as e:
            # Failure
            llm_requests.labels(
                tenant_id=tenant_id,
                model=config.model,
                status='error'
            ).inc()
            raise
```

```
# Update quality metrics
async def update_rag_quality_metrics(tenant_id: str):
    evaluator = TenantEvaluator(tenant_id)
    scores = await evaluator.evaluate_qa_quality(test_cases)

    rag_quality.labels(tenant_id=tenant_id, metric='faithfulness').set(
        scores['faithfulness']
    )
    rag_quality.labels(tenant_id=tenant_id, metric='relevancy').set(
        scores['answer_relevancy']
    )
```

Grafana Dashboards

Tenant-specific dashboard:

```
# Grafana dashboard queries
panels:
- title: "LLM Requests per Tenant"
  query: |
    sum(rate(llm_requests_total[5m])) by (tenant_id)

- title: "P95 Latency by Tenant"
  query: |
    histogram_quantile(0.95,
      sum(rate(llm_request_duration_seconds_bucket[5m])) by (tenant_id, le)
    )

- title: "Token Usage by Tenant"
  query: |
    sum(llm_tokens_total) by (tenant_id)

- title: "RAG Quality Score"
  query: |
    rag_quality_score{metric="faithfulness"} by (tenant_id)

- title: "Error Rate by Tenant"
  query: |
    sum(rate(llm_requests_total{status="error"}[5m])) by (tenant_id)
    /
    sum(rate(llm_requests_total[5m])) by (tenant_id)
```

Tenant Comparison Dashboard:

```
# Compare all tenants
@app.get("/api/admin/tenant-comparison")
async def tenant_comparison():
    tenants = db.collection("tenants").find()
```

```

comparison = []
for tenant in tenants:
    tenant_id = tenant["id"]

    comparison.append({
        "tenant_id": tenant_id,
        "requests_per_day": get_requests_per_day(tenant_id),
        "avg_latency_ms": get_avg_latency(tenant_id) * 1000,
        "tokens_per_day": get_tokens_per_day(tenant_id),
        "cost_per_day": calculate_cost(tenant_id),
        "rag_quality": get_rag_quality_score(tenant_id),
        "error_rate": get_error_rate(tenant_id)
    })

return comparison

```

Alerting Per Tenant

```

# Prometheus alerts
groups:
- name: tenant_alerts
  rules:
    # High latency for specific tenant
    - alert: TenantHighLatency
      expr: |
        histogram_quantile(0.95,
          sum(rate(llm_request_duration_seconds_bucket[5m])) by (tenant_id, le)
        ) > 5
      for: 10m
      labels:
        severity: warning
      annotations:
        summary: "Tenant {{ $labels.tenant_id }} experiencing high latency"
        description: "P95 latency is {{ $value }}s"

    # Quota exceeded
    - alert: TenantQuotaExceeded
      expr: |
        sum(llm_tokens_total) by (tenant_id) > 1000000
      labels:
        severity: critical
      annotations:
        summary: "Tenant {{ $labels.tenant_id }} exceeded monthly quota"

    # Low RAG quality
    - alert: TenantLowRAGQuality
      expr: |
        rag_quality_score{metric="faithfulness"} < 0.7
      for: 1h

```

```

labels:
  severity: warning
annotations:
  summary: "Tenant {{ $labels.tenant_id }} RAG quality degraded"
  description: "Faithfulness score: {{ $value }}"

# High error rate
- alert: TenantHighErrorRate
  expr: |
    sum(rate(llm_requests_total{status="error"}[5m])) by (tenant_id)
    /
    sum(rate(llm_requests_total[5m])) by (tenant_id)
    > 0.1
  for: 5m
  labels:
    severity: critical
  annotations:
    summary: "Tenant {{ $labels.tenant_id }} error rate > 10%"

```

Cost Tracking Per Tenant

```

# Calculate LLM costs
def calculate_tenant_cost(tenant_id: str, month: str):
    # Get usage
    usage = db.collection("llm_usage").aggregate([
        {
            "$match": {
                "tenant_id": tenant_id,
                "month": month
            }
        },
        {
            "$group": {
                "_id": "$provider",
                "total_tokens": {"$sum": "$tokens"}
            }
        }
    ])

    # Pricing per provider
    pricing = {
        "openai": {
            "gpt-4": {"input": 0.03, "output": 0.06}, # per 1K tokens
            "gpt-3.5-turbo": {"input": 0.001, "output": 0.002}
        },
        "anthropic": {
            "claude-3-5-sonnet": {"input": 0.003, "output": 0.015}
        },
        "groq": {
            "llama-3.3-70b": {"input": 0.0, "output": 0.0} # Free tier

```

```

    }
}

total_cost = 0
for record in usage:
    provider = record["_id"]
    tokens = record["total_tokens"]

    # Estimate cost (assuming 50/50 input/output)
    input_tokens = tokens * 0.5
    output_tokens = tokens * 0.5

    config = TENANT_CONFIGS[tenant_id]
    model_pricing = pricing[provider][config.model]

    cost = (
        (input_tokens / 1000) * model_pricing["input"] +
        (output_tokens / 1000) * model_pricing["output"]
    )

    total_cost += cost

return total_cost

# Billing API
@app.get("/api/billing/{month}")
async def get_billing(request: Request, month: str):
    tenant_id = request.state.tenant_id

    cost = calculate_tenant_cost(tenant_id, month)
    usage = get_token_usage(tenant_id, month)

    return {
        "tenant_id": tenant_id,
        "month": month,
        "total_tokens": usage,
        "cost_usd": cost,
        "breakdown": get_cost_breakdown(tenant_id, month)
    }

```

Complete Multi-Tenant LLM System Example

```

# main.py - Complete multi-tenant LLM/RAG system

from fastapi import FastAPI, Request, HTTPException, Depends
from typing import Dict, List
import anthropic
import groq
from langchain.vectorstores import Pinecone
from langchain.embeddings import OpenAIEmbeddings

```

```

from prometheus_client import Counter, Histogram, make_asgi_app
import pinecone

app = FastAPI()

# Prometheus metrics endpoint
metrics_app = make_asgi_app()
app.mount("/metrics", metrics_app)

# Metrics
llm_requests = Counter('llm_requests_total', 'Total requests', ['tenant_id',
'status'])
llm_latency = Histogram('llm_latency_seconds', 'Latency', ['tenant_id'])
tokens_used = Counter('tokens_used_total', 'Tokens', ['tenant_id'])

# Tenant configurations
TENANT_CONFIGS = {...} # From earlier

# Tenant extraction middleware
@app.middleware("http")
async def tenant_middleware(request: Request, call_next):
    # Extract from subdomain: tenant-a.api.example.com
    hostname = request.url.hostname
    if hostname.endswith(".api.example.com"):
        tenant_id = hostname.split(".")[0]
    # Or from JWT
    elif "Authorization" in request.headers:
        token = request.headers["Authorization"].replace("Bearer ", "")
        tenant_id = decode_jwt(token)["tenant_id"]
    # Or from header
    elif "X-Tenant-ID" in request.headers:
        tenant_id = request.headers["X-Tenant-ID"]
    else:
        raise HTTPException(401, "Tenant not identified")

    request.state.tenant_id = tenant_id
    response = await call_next(request)
    return response

# LLM endpoint
@app.post("/api/chat")
async def chat(request: Request, message: str):
    tenant_id = request.state.tenant_id
    config = TENANT_CONFIGS[tenant_id]

    with llm_latency.labels(tenant_id=tenant_id).time():
        try:
            # Route to provider
            if config.provider == "groq":
                client = groq.AsyncGroq(api_key=config.api_key)
                response = await client.chat.completions.create(
                    model=config.model,
                    messages=[
                        {"role": "system", "content": config.system_prompt},

```

```

        {"role": "user", "content": message}
    ]
)
result = response.choices[0].message.content
tokens = response.usage.total_tokens

# Track metrics
llm_requests.labels(tenant_id=tenant_id, status='success').inc()
tokens_used.labels(tenant_id=tenant_id).inc(tokens)

# Log for billing
await log_usage(tenant_id, config.provider, tokens)

return {"response": result}

except Exception as e:
    llm_requests.labels(tenant_id=tenant_id, status='error').inc()
    raise HTTPException(500, str(e))

# RAG endpoint
@app.post("/api/rag/query")
async def rag_query(request: Request, question: str):
    tenant_id = request.state.tenant_id

    # Get tenant's vector store
    embeddings = OpenAIEmbeddings()
    vectorstore = Pinecone.from_existing_index(
        index_name="multi-tenant-kb",
        embedding=embeddings,
        namespace=tenant_id
    )

    # Retrieve context
    docs = vectorstore.similarity_search(question, k=5)
    context = "\n".join([doc.page_content for doc in docs])

    # Generate with context
    augmented_prompt = f"""Context:
{context}

Question: {question}

Answer based on the context above:"""

    response = await chat(request, augmented_prompt)
    return response

# Admin: View all tenants
@app.get("/api/admin/tenants")
async def list_tenants():
    tenants = []
    for tenant_id, config in TENANT_CONFIGS.items():
        tenants.append({
            "tenant_id": tenant_id,

```

```
        "provider": config.provider,
        "model": config.model,
        "monthly_quota": config.monthly_quota,
        "usage_this_month": get_monthly_usage(tenant_id)
    })
    return tenants

# Health check
@app.get("/health")
async def health():
    return {"status": "healthy"}

if __name__ == "__main__":
    import uvicorn
    uvicorn.run(app, host="0.0.0.0", port=8000)
```

Deployment Architecture

Kubernetes Multi-Tenant Setup

```
# namespace-per-tenant.yaml
apiVersion: v1
kind: Namespace
metadata:
  name: tenant-a
  labels:
    tenant: tenant-a

---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: llm-service
  namespace: tenant-a
spec:
  replicas: 2
  selector:
    matchLabels:
      app: llm-service
      tenant: tenant-a
  template:
    metadata:
      labels:
        app: llm-service
        tenant: tenant-a
    spec:
      containers:
        - name: api
          image: llm-service:latest
          env:
```

```
- name: TENANT_ID
  value: "tenant-a"
- name: LLM_PROVIDER
  value: "groq"
- name: LLM_API_KEY
  valueFrom:
    secretKeyRef:
      name: tenant-a-llm-secret
      key: api-key
resources:
  requests:
    memory: "512Mi"
    cpu: "500m"
  limits:
    memory: "1Gi"
    cpu: "1000m"

---
apiVersion: v1
kind: Service
metadata:
  name: llm-service
  namespace: tenant-a
spec:
  selector:
    app: llm-service
    tenant: tenant-a
  ports:
    - port: 80
      targetPort: 8000

---
# Resource quotas per tenant
apiVersion: v1
kind: ResourceQuota
metadata:
  name: tenant-quota
  namespace: tenant-a
spec:
  hard:
    requests.cpu: "4"
    requests.memory: 8Gi
    limits.cpu: "8"
    limits.memory: 16Gi
    persistentvolumeclaims: "10"
```

Security Considerations

1. Data Isolation

```
# Prevent cross-tenant data access
@app.get("/api/documents/{doc_id}")
async def get_document(request: Request, doc_id: str):
    tenant_id = request.state.tenant_id

    # Fetch document
    doc = db.collection("documents").find_one({"id": doc_id})

    # CRITICAL: Verify ownership
    if doc["tenant_id"] != tenant_id:
        raise HTTPException(403, "Access denied")

    return doc
```

2. API Key Management

```
# Store tenant API keys securely
import boto3
from cryptography.fernet import Fernet

def get_tenant_api_key(tenant_id: str, provider: str):
    # Fetch from AWS Secrets Manager
    client = boto3.client('secretsmanager')

    secret_name = f"{tenant_id}/{provider}/api-key"
    response = client.get_secret_value(SecretId=secret_name)

    # Decrypt
    encrypted_key = response['SecretString']
    key = decrypt(encrypted_key)

    return key
```

3. Rate Limiting Per Tenant

```
from slowapi import Limiter
from slowapi.util import get_remote_address

limiter = Limiter(key_func=lambda: request.state.tenant_id)

@app.post("/api/chat")
@limiter.limit("100/minute") # Per tenant
async def chat(request: Request, message: str):
    # ...
```

Unit Tests

```
import pytest
from fastapi.testclient import TestClient

@pytest.fixture
def tenant_a_client():
    client = TestClient(app)
    client.headers.update({"X-Tenant-ID": "tenant_a"})
    return client

@pytest.fixture
def tenant_b_client():
    client = TestClient(app)
    client.headers.update({"X-Tenant-ID": "tenant_b"})
    return client

def test_data_isolation(tenant_a_client, tenant_b_client):
    # Tenant A creates document
    response_a = tenant_a_client.post("/api/documents", json={"title": "Doc A"})
    doc_id = response_a.json()["id"]

    # Tenant B should NOT be able to access
    response_b = tenant_b_client.get(f"/api/documents/{doc_id}")
    assert response_b.status_code == 403

def test_separate_rag_contexts(tenant_a_client, tenant_b_client):
    # Each tenant should only see their docs
    response_a = tenant_a_client.post("/api/rag/query", json={"question": "test"})
    response_b = tenant_b_client.post("/api/rag/query", json={"question": "test"})

    # Should get different answers based on their knowledge bases
    assert response_a.json() != response_b.json()
```

Pattern Comparison

Pattern	Isolation	Cost	Complexity	Scalability	Best For
Database per Tenant	★★★★★	\$\$\$\$\$	★★★★	★★★	Enterprise, compliance
Row-Level	★★	\$	★★	★★★★★	Many small tenants
Schema per Tenant	★★★★	\$\$\$	★★★	★★★★	Medium tenants
Service per Tenant	★★★★★	\$\$\$\$\$	★★★★★	★★	VIP customers
Shared with Namespace	★★★	\$\$	★★	★★★★	Standard SaaS

Interview Answer Template

"How would you design a multi-tenant LLM system?"

Answer Framework:

1. Define Tenancy Model

- "I'd use row-level multi-tenancy for data (single DB with tenant_id)"
- "For LLM, each tenant gets their own config (model, provider, quotas)"
- "For RAG, use namespaced vector store (Pinecone namespaces or separate Chroma collections)"

2. Tenant Identification

- "Extract tenant_id from subdomain (tenant-a.api.example.com)"
- "Or from JWT token claims"
- "Propagate via X-Tenant-ID header to all microservices"

3. Data Isolation

- "All queries filter by tenant_id"
- "RAG uses tenant-specific namespaces"
- "Verify tenant ownership before returning data"

4. Resource Management

- "Track token usage per tenant"
- "Enforce monthly quotas"
- "Different LLM tiers (Free: Groq, Premium: GPT-4)"

5. Monitoring

- "Prometheus metrics with tenant_id label"
- "Per-tenant dashboards in Grafana"
- "Alert on quota exceeded, high latency, low quality"

6. Cost Tracking

- "Log every LLM call with tokens used"
- "Calculate cost per tenant per month"
- "Billing API for invoicing"

Example: "For a customer support chatbot platform:

- 1000 small tenants → row-level DB, shared LLM service
- Each tenant has their own RAG knowledge base (Pinecone namespace)
- Free tier: Groq (100K tokens/month)
- Paid tier: GPT-4 (1M tokens/month)
- Monitor quality per tenant with RAGAS metrics
- Alert if faithfulness score drops below 0.7"

Best Practices Summary

Data Isolation

- ☒ Always filter by tenant_id in queries
- ☒ Use database constraints to enforce isolation
- ☒ Verify tenant ownership before returning data
- ☒ Never trust client-provided tenant_id alone

LLM/RAG

- ☒ Separate vector stores per tenant (namespaces)
- ☒ Tenant-specific system prompts
- ☒ Track usage and enforce quotas
- ☒ Don't share API keys across tenants

Monitoring

- ☒ Add tenant_id to all metrics
- ☒ Per-tenant dashboards
- ☒ Alert on quota/quality issues
- ☒ Don't aggregate metrics across all tenants only

Cost

- ☒ Track token usage per tenant
- ☒ Different pricing tiers
- ☒ Usage-based billing
- ☒ Don't use same LLM for all tenants

Security

- ☒ Encrypt API keys
- ☒ Use secrets manager (AWS Secrets, Vault)
- ☒ Rate limit per tenant
- ☒ Don't log tenant API keys

Testing

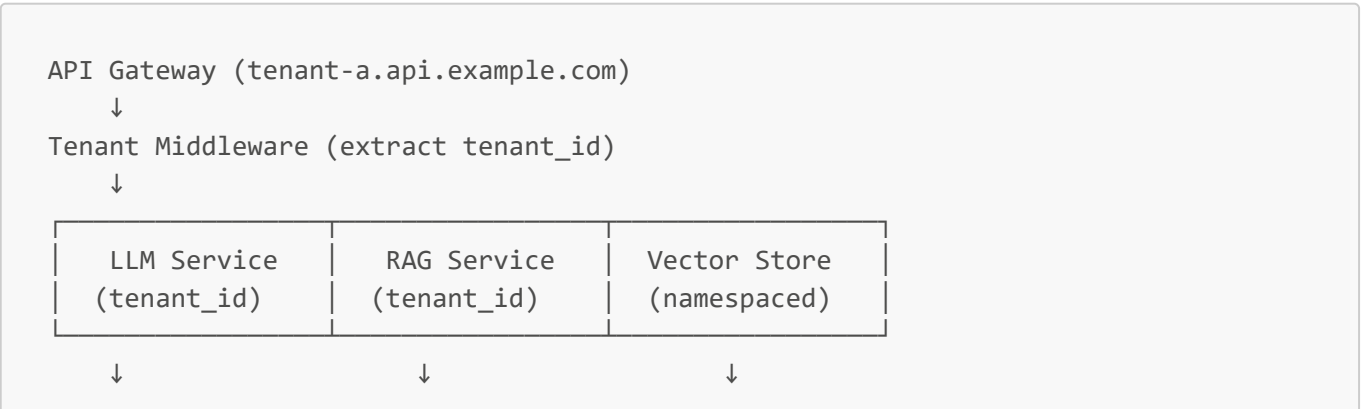
- ☒ Test cross-tenant isolation
- ☒ Load test per tenant
- ☒ Test quota enforcement
- ☒ Don't test with production tenant data

Real-World Example: Customer Support Platform

Requirements:

- 500 companies (tenants)
- Each has their own knowledge base
- AI-powered support chatbot
- Track quality and costs per tenant

Architecture:



PostgreSQL (tenant_id col)	Pinecone (namespace)	Prometheus (tenant_id label)
-------------------------------	-------------------------	---------------------------------

Implementation:

- 1. **Free tier** (100 tenants): Groq, 50K tokens/month, basic RAG
- 2. **Standard** (300 tenants): GPT-3.5, 500K tokens/month, enhanced RAG
- 3. **Premium** (100 tenants): GPT-4, 2M tokens/month, custom prompts

Monitoring:

- Response quality (RAGAS): faithfulness > 0.8
- Latency: P95 < 2s
- Cost: Track per tenant, bill monthly
- Alerts: Quota exceeded, quality drop, high errors

Results:

- Cost per tenant: \$10-500/month (based on tier)
- 99.5% uptime SLA
- Avg response quality: 0.85 faithfulness
- 2M requests/day across all tenants

Tools and Technologies

Multi-Tenancy:

- PostgreSQL (Row-Level Security)
- MongoDB (tenant_id field)
- Kubernetes (namespaces)

LLM:

- OpenAI, Anthropic, Groq
- LangChain (RAG orchestration)
- LlamaIndex (alternative)

Vector Stores:

- Pinecone (namespaces)
- Chroma (collections)
- Weaviate (tenant filtering)

Monitoring:

- Prometheus (metrics)
- Grafana (dashboards)
- Jaeger (distributed tracing)

Evaluation:

- RAGAS (RAG quality)
- LangSmith (LLM observability)
- Weights & Biases (experiment tracking)

Cost Tracking:

- Custom billing service
 - Stripe (payment processing)
 - AWS Cost Explorer
-

Summary

Multi-tenancy = Shared infrastructure, isolated data

Key Patterns:

1. Database per tenant (highest isolation)
2. Row-level with tenant_id (lowest cost)
3. Schema per tenant (middle ground)

For LLM/RAG:

- Tenant-specific configs and quotas
- Namespaced vector stores
- Per-tenant evaluation
- Usage tracking and billing

Monitoring:

- Metrics with tenant_id labels
- Per-tenant dashboards
- Quality alerts
- Cost tracking

Start simple (row-level) → **Scale to dedicated** (database-per-tenant) for large customers